MeLi Challenge Operación Fuego de Quasar

Es un proyecto creado a partir del challenge que me propuso Mercado Libre para mi postulación laboral. Las instrucciones completas se encuentran en Quasar

Comenzando

Este diagrama muestra de manera general cómo interactúan los paquetes del proyecto.

Store: se utiliza para guardar los datos en memoria. Los stores son inicializados con los datos de los tres satélites conocidos.

View: representa la estructura de los request y los response

Service: tiene la lógica de negocio para obtener el mensaje y la ubicación del emisor del mensaje Model: representa las entidades del negocio


Diagrama

Pre-requisitos

Tener Golang instalado para probar localmente Link y Postman para hacer las peticiones

Ejecución

Se puede clonar y ejecutar en el entorno Local o consumir los endpoints deployados a través del Cloud Heroku . Dado que es un proyecto privado, generé un token en GitHub para poder clonarlo localmente .

a) **Entorno local**

Para ejecutarlo en local debemos posicionarnos en la rama develop ya que destiné la rama main para el deploy en Heroku

1- Clonar el repositorio:

```
https://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@github.com/marciobagnus/quasar.git
```

2- Abrir un git bash en la raíz del proyecto y posicionarse en la rama develop:

```
git checkout develop
```

3- Ejecutar el comando:

```
go run main.go
```

(En caso de una ventana emergente de firewall, permitir el acceso)

b) **Cloud**

Se encuentra deployado en https://thawing-earth-38148.herokuapp.com/

Endpoints

•Topsecret

-Método: POST

-Descripción: recibe por body un JSON con una key “satellites” que contiene los 3 satélites. Cada satélite contiene su nombre, distancia al emisor y el mensaje. Responde un JSON con el mensaje descifrado y la posición del emisor.

-URL: https://thawing-earth-38148.herokuapp.com/topsecret/ ó http://localhost:8080/topsecret/

-Ejemplo:

```
{
    "satellites": [
        {
            "Name":     "Kenobi",
            "Distance": 1000,
            "Message":  ["este", "", "", "mensaje",""]

        },
        {
            "Name":     "skywalker",
            "Distance": 188.5,
            "Message":  ["", "es", "", "", "secreto"]

        },

                {
            "Name":     "sato",
            "Distance": 1992.7,
            "Message":  ["este", "", "un", "", ""]

        }

    ]
}
```

•TopSecret_Split

Este endpoint tiene dos métodos implementados:

1-POST

-Descripción: Recibe por body la distancia al emisor y el mensaje. Debe enviarse el nombre del satélite en la URL. Devuelve un mensaje avisando si se guardo o no el satélite. Se encarga de guardar en el store un satélite.

-URL: https://thawing-earth-38148.herokuapp.com/topsecret_split/{nombre} ó http://localhost:8080/topsecret_split/{nombre}

-Ejemplo:

```
http://localhost:8080/topsecret_split/sato
{
"distance": 100.0,
"message": ["este", "", "", "mensaje", ""]
}
```

2-GET

-Descripción: devuelve el mensaje descifrado y la posición del emisor.

-URL: https://thawing-earth-38148.herokuapp.com/topsecret_split/ ó http://localhost:8080/topsecret_split/

Despliegue

Para deployarlo en Heroku segui estos Pasos

Construido con

- Lenguage: Golang
- IDE: Visual Studio Code
- Versionado: Git Flow
- Repositorio: GitHub
- Cloud : Heroku

Notas

No llegue a implementar todo lo que hubiese querido por una cuestión de entregarlo en un tiempo razonable. El proyecto funciona bien en el entorno local pero al deployarlo con Heroku tiene algunas fallas que intentaré resolver.

Autores

- **Marcio Bagnus** - marciobagnus

Experiencia

Al través de todo el proyecto fue un impedimento no saber Goolang, pero aprendiéndolo y con el correr de los días lo fui superando. En un primer momento me concentré en investigar el modelo matemático que respondía al primer método y descubrí la trilateración, que me permitió aproximarme a la resolución. Luego con el método del mensaje intenté buscarle el mismo lado matemático, pero resulto ser mas intuitivo. Si bien

no había una fecha para entregar el proyecto, busqué un equilibrio entre entregarlo en un plazo razonable y resolverlo bien. Me gustó haberme sentido desafiado y aprender Goolang.

Mejoras 💥

- Manejo de errores entre las capas mas completo
- Utilizar Azure para CI-CD
- Documentar la API con Swagger
- Crear variables de entorno
- Testear

Agradecimientos 🎁

- A mi familia y amigos que me contuvieron en estos 10 días ♡
- A mi amigo Lucas Cunibertti que me ayudó con otra visión sobre el algoritmo del segundo método 😊